

Laboratory work 5

1 Objectives

The objective of this laboratory is to use the vectors and matrixes operations defined in previous laboratories, together with the transformation matrixes, in order to perform different transformations on a 2D figure.

2 Using an SDL application

In order to exemplify graphically the 2D transformations, we will use an application very similar to the one from Laboratory 1, based on the SDL (Simple DirectMedia Layer) library.

3 SDL_Renderer

SDL_Renderer is a structure that handles all rendering. It is tied to an **SDL_Window** so it can only render within that **SDL_Window**. It also keeps track of the settings related to the rendering.

There are several important functions tied to the **SDL_Renderer**:

1. **SDL_SetRenderDrawColor** sets the color that will be used in all drawing operations, until another call to the function is performed.

```
SDL_SetRenderDrawColor(renderer, r, g, b, a);
```

2. **SDL_RenderClear** clears the entire window display area using the current active color, previously set with **SDL_SetRenderDrawColor** function.

```
SDL_RenderClear(renderer);
```

3. **SDL_RenderPresent** will display everything drawn in the renderer to the screen. Until this function is called, all the drawing takes place in a hidden buffer that is not visible to the user. The call to **SDL_RenderPresent** should be made only once, after all the drawing functions have been called.

```
SDL_RenderPresent(renderer);
```

If we want to draw a rectangle, we need to specify the starting position of the rectangle (the x and y coordinates) and the dimension (width and height). In order to specify the color we are using the **SDL_Color** structure using the Red, Green, Blue and Alpha values. The **SDL_RenderFillRect()** function will draw the rectangle with the previously specified color.

```
SDL_FRect rectangleCoordinates = {100, 100, 200, 200};
SDL_Color rectangleColor { 255, 0, 0, 255 };
SDL_SetRenderDrawColor(renderer,
    rectangleColor.r, rectangleColor.g, rectangleColor.b, rectangleColor.a);
```

```
SDL_RenderFillRect(renderer, &rectangleCoordinates);
```

For each color channel we specify the values between 0 and 255. Look at the following table for some color channel values, by combining the channel values we get different colors.

Color name	Red channel	Green channel	Blue channel	Color
White	255	255	255	
Red	255	0	0	
Green	0	255	0	
Blue	0	0	255	
Yellow	255	255	0	
Black	0	0	0	

If we want to draw a line, we must specify its start and end point.

```
vec3 p1(100, 100, 1), p2(400, 100, 1);  
  
SDL_RenderLine(renderer, p1.x, p1.y, p2.x, p2.y);
```

4 Assignment

Download and run the application from the laboratory website. Try to understand the basic example and then extend the application with the following functionality:

- Include into the application your implementation files (.cpp).
- Define an initial rectangle with the top-left corner in P1(100, 100) and bottom-right corner in P2(400, 200).
- Rotate the rectangle around its center (diagonals intersection) by 10 degrees clockwise when RIGHT_ARROW is pressed and 10 degrees counterclockwise when LEFT_ARROW is pressed.
- Scale the rectangle having the top-left corner as a reference, using UP_ARROW and DOWN_ARROW keys.